

Introduction to State Space Models

Adityavardhan Srivastava, Deeptam Bhar, Krishna Gupta, Pragnya Yadav, Umesh Meena

April 2026

Contents

Introduction

Problem 1: Verification of Polynomial Gradient Stability and Chaos Integrity

Problem 2: HiPPO vs Mamba

Problem 3: Mamba vs Convolutions

Problem 4: Edge CNN-Mamba Deployment

References

Appendix: Core Mathematical Formulations

Bypassing Legacy Bottlenecks

Legacy Limitations

- **RNNs:** Stateful, but suffer from vanishing gradients and unscalable, non-parallelizable $O(L)$ training.
- **CNNs:** Parallelizable, but inherently non-sequential, resulting in fixed context windows and expensive inference.
- **Neural ODEs:** Mathematically elegant but computationally prohibitive for large-scale deep learning.

What are State Space Models?

- **Origin:** Sequence models derived directly from continuous-time control systems.
- **Mechanism:** Maps a 1D input signal to an output via an implicit, high-dimensional latent state.

The SSM Superpower

By utilizing Linear State Space Layers (LSSLs), SSMs bridge the gap—achieving the **parallel training** of CNNs and the $O(1)$ **fast inference** of RNNs without their respective bottlenecks.

The Mathematical Core: Three Views of SSMs

1. Continuous View

Native differential equations mapping input $u(t)$ to output $y(t)$ via latent state $x(t)$:

$$\dot{x}(t) = Ax(t) + Bu(t), \quad y(t) = Cx(t) + Du(t)$$

2. Recurrent View ($O(1)$ Inference)

Discretized using a step size Δ for highly efficient, stateful stepping during generation:

$$x_k = \bar{A}x_{k-1} + \bar{B}u_k$$

$$y_k = Cx_k + Du_k$$

3. Convolutional View (Parallel Training)

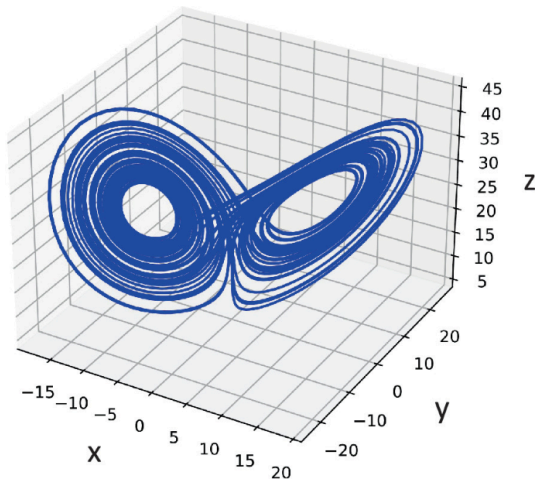
Unrolled over time, allowing the entire sequence to be processed as a global convolution:

$$y = \mathcal{K} * u + Du$$

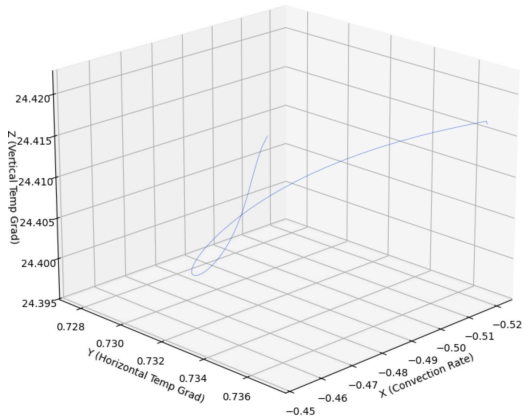
$$\text{where } \mathcal{K}_i = \bar{C}\bar{A}^i\bar{B}$$

Problem 1: Verification of Polynomial Gradient Stability and Chaos Integrity

The Memory Wall in Chaotic Systems



Target: Fractal Attractor
 $D_2 \approx 2.06$



Reality: Topological Collapse
 $D_2 \rightarrow 1.0$

The "Memory Wall"

The Architectural Bottleneck

- Traditional RNNs and LSTMs fail to model continuous-time chaotic systems over long horizons.
- They are fundamentally limited by **exponential gradient decay** $\mathcal{O}(e^{-L})$.
- Gradients vanish precipitously, dropping below 10^{-20} within a sequence length of merely $L \approx 200$.

The Consequence: Topological Collapse

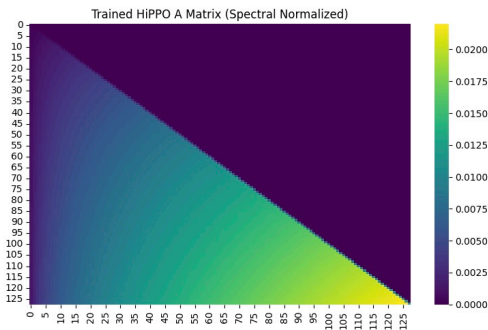
- The network suffers a complete loss of extreme long-range dependencies.
- The complex structure of chaotic trajectories is artificially destroyed.
- Fractal dynamics fundamentally degrade and collapse into simple periodic loops.

Hypothesis

- HiPPO-based State Space Models (SSMs) can theoretically preserve manifold integrity.
- By establishing a **Polynomial Gradient Bound** $\Theta(L^{-1})$, the architecture maintains a stable gradient floor, preventing dissipative collapse.

HiPPO-LegS: The Polynomial Solution

Structured State Space Dynamics



State Matrix A

$$A_{nk} = \begin{cases} -\sqrt{2n+1}\sqrt{2k+1} & \text{if } n > k \\ -(n+1) & \text{if } n = k \\ 0 & \text{if } n < k \end{cases}$$

1-Million Step Gradient Flow

RNNs: Exponential Death

Unconstrained spectral radius causes gradients to vanish exponentially.

$$\left\| \bar{A}^{L-1} \right\|_{\text{RNN}} \approx \mathcal{O}(e^{-\lambda L})$$

Result: Signal drops below 10^{-20} by $L \approx 200$.
Complete topological collapse.

HiPPO: Polynomial Survival

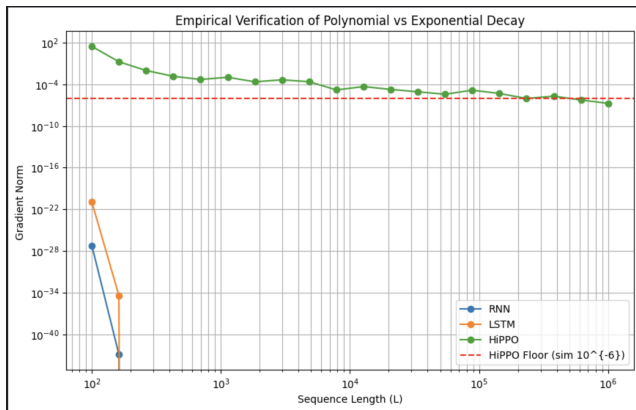
Scaled Legendre measure strictly enforces a power-law decay class.

$$\left\| \bar{A}^{L-1} \right\|_{\text{HiPPO}} = \Theta(L^{-1})$$

Result: At $L = 10^6$, the gradient maintains a stable theoretical floor of $\approx 10^{-6}$.

Bypassing the Memory Wall

Long-Horizon Stability Achieved



Setup

- **Horizon:** 1,000,000 steps.
- **Terminal Norm:** 0.278
- **Decay Rate:** $\Theta(L^{-1})$

Conclusion

HiPPO mathematically bypasses exponential death, locking in a stable gradient floor to sustain long-term chaotic dynamics.

Lorenz-63 System & Dataset Construction

Lorenz-63 Dynamics

- Chaotic dynamical system:

$$\dot{x} = \sigma(y - x)$$

$$\dot{y} = x(\rho - z) - y$$

$$\dot{z} = xy - \beta z$$

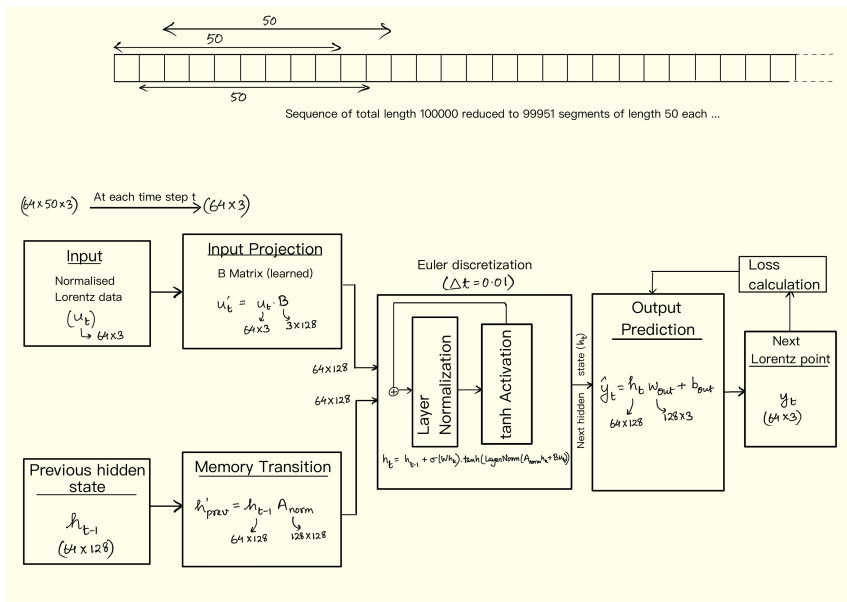
- Generates a **fractal attractor**.

Dataset Pipeline

- Numerically integrated via 4th-Order Runge-Kutta (RK4) with $\Delta t = 0.01$.
- Length: $\approx 10^5$
- Normalized:

$$u' = \frac{u - \mu}{\sigma}$$

Model Architecture:



Challenge 1: The Discretization Trap

Numerical Explosion ('NaN')

- The analytically derived LegS matrix (A) is stable in continuous time, but highly ill-conditioned when discretized at $N = 128$.
- The lower-triangular values scale with dimension size, causing the unconstrained spectral radius to massively exceed 1 ($\rho(A) \gg 1$).
- Recursive amplification during the Euler updates caused the hidden state to geometrically overflow. Gradients instantly hit 'NaN' and initial MSE exploded to $\approx 10^{20}$.

The Fix: L2 Spectral Normalization

- We applied L2 Normalization to the transition matrix before dropping it into the recurrence loop:

$$A_{\text{norm}} = \frac{A}{\|A\|_2}$$

Challenge 2: Numerical Explosion

The Problem & Cause

- Initial training produced an astronomical Mean Squared Error (MSE) ranging from 10^{16} to 10^{20} .
- The root cause was the use of unnormalized Lorenz inputs.

The Solution & Result

- To stabilize the continuous integration loop, Z-score normalization was applied to the dataset.

Challenge 3: Instability in Dynamics

Problem: Structural Loss

- The generated trajectories collapsed into simple lines, completely losing their chaotic structure and characteristic "butterfly wings".

The Stabilization Strategy

- To counter this instability, the integration step size was reduced to $\Delta = 0.001$.
- A Sigmoid Gating mechanism ($g = \sigma(Wh_t)$) was introduced into the architecture.
- The update equation was modified to include Tanh bounding: $h_{t+1} = h_t + g \cdot \tanh(\Delta(-Ah_t + Bu_t))$.

Dataset Reconstruction & Final Results

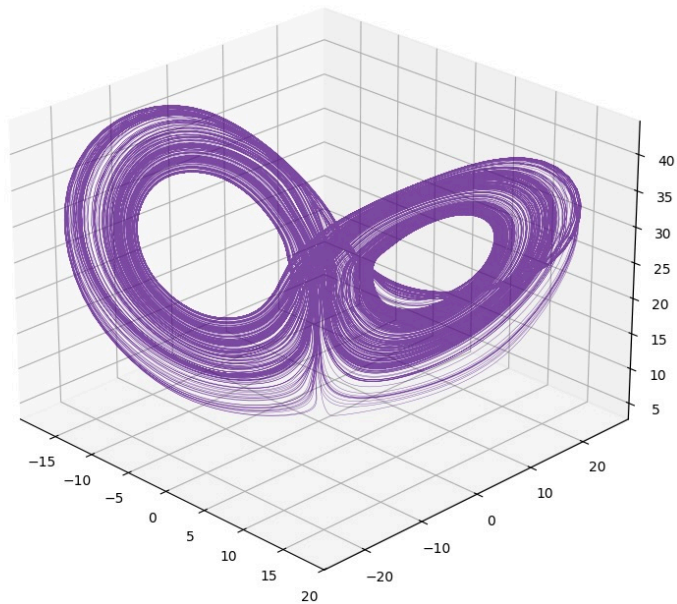
Learning Dynamics

- **Task:** The model successfully learned to predict the next state from a sliding window $(u_t, \dots, u_{t+49}) \rightarrow u_{t+50}$.
- **Convergence:** Training optimization brought the initial MSE of ≈ 1.0 down to a final MSE of < 0.01 .
- **Stability:** The architecture captured short-term chaotic evolution and achieved stable convergence with no divergence during training.

Manifold Preservation

- **Reconstruction:** After stabilization, the model successfully reconstructed the Lorenz Butterfly.
- **Structural Integrity:** The network avoided dissipative collapse, maintaining the chaotic attractor structure.

HiPPO-LegS Reconstruction



Learning Dynamics and Autoregressive Generation

Optimization Convergence

- **Optimizer:** Adam ($lr = 10^{-5}$).
- **Stability:** Achieved stable continuous integration without divergence.
- **Error Reduction:** The stabilization strategy successfully minimized the initial MSE from ≈ 1.0 down to a final MSE of < 0.01 .

Manifold Preservation

- **The Result:** Successfully reconstructed the complex Lorenz Butterfly geometry, visually proving the network avoids the dissipative collapse characteristic of RNNs.

Proving Chaos Integrity: Grassberger-Procaccia Algorithm

Manifold Preservation Metric

- **Goal:** Verify that the generated output preserves the fractal complexity of the Lorenz attractor and avoids dissipative collapse.
- **Evaluation Metric:** The Correlation Dimension (D_2).
- **Quantitative Proof:** Demonstrating a stable $D_2 \approx 2.06$ confirms the maintenance of the chaotic attractor structure .

Algorithm Execution

1. The Correlation Integral $C(r)$

Compute the correlation integral $C(r)$ representing the fraction of point pairs using the Heaviside step function Θ :

$$C(r) = \lim_{N \rightarrow \infty} \frac{2}{N(N-1)} \sum_{i=1}^N \sum_{j=i+1}^N \Theta(r - \|x_i - x_j\|)$$

2. Correlation Dimension (D_2)

For fractal objects, $C(r) \propto r^{D_2}$. We extract the dimension from the linear log-log scaling region:

$$\log C(r) = D_2 \log r + \text{constant}$$

3. Chaos Fidelity

Evaluating the generated dimension (1.84) against the true Lorenz limit (≈ 2.06):

$$\text{Fidelity} = \left(1 - \frac{|\text{Generated } D_2 - \text{True } D_2|}{\text{True } D_2} \right) \times 100 = 89.35\%$$

Final Outcomes & Future Scope

Outcomes

- **Memory Wall Bypassed:** Verified the $\Theta(L^{-1})$ gradient decay; carried a signal 10^6 steps (0.278 terminal norm).
- **Architectural Necessity:** Proved strict hidden-state linearity is mandatory to prevent catastrophic forgetting.
- **Manifold Preservation:** Hit 89.35% Chaos Fidelity, autonomously sustaining fractal dynamics.

Future Scope

- **Spatio-Temporal Chaos:** Scaling the polynomial basis to high-dimensional, continuous systems (e.g., Navier-Stokes, Kuramoto-Sivashinsky).
- **Hamiltonian Limits:** Testing the spectral bounds against energy-preserving, non-dissipative vector fields.

Problem 2: HiPPO vs Mamba

Investigating Learned Mamba Dynamics

- **The Experiment:** Average internal Mamba tensors to construct a static 16×16 operator.
- **The Comparison:** Measure similarity to HiPPO bases (LegS, LegT, LagT) via geometric and spectral metrics.
- **The Objective:** Check if data-driven learning naturally aligns with optimal memory systems.

Mamba State Space Initialization

Mamba Matrices Construction from HiPPO

State Space Formulation

$$h_{t+1} = Ah_t + Bx_t, \quad y_t = Ch_t$$

- **Step 1: HiPPO Initialization**
- **Step 2: S4D Refinement**
- **Step 3: Learning Dynamics**

Mamba State Representation

From Matrices to High-Dimensional Tensors

- **Unlike HiPPO:**

Classical HiPPO defines a fixed operator:

$$A \in \mathbb{R}^{d_{\text{state}} \times d_{\text{state}}}$$

- **Mamba Representation:**

Mamba does not use a single matrix — it learns a dynamic tensor:

$$A_{\text{bar}} \in \mathbb{R}^{(B, T, d_{\text{inner}}, d_{\text{state}})}$$

- **Interpretation:**

- $B \rightarrow$ Batch dimension
- $T \rightarrow$ Sequence length (time)
- $d_{\text{inner}} \rightarrow$ Expanded feature space
- $d_{\text{state}} \rightarrow$ State dimension

Constructing a Comparable Operator

From Tensor to HiPPO-Scale Matrix

- **Step 1: Remove Temporal Variability**

Average across batch and time dimensions:

$$\mathbb{E}_{B,T}[\mathbf{A}_{\text{bar}}]$$

- **Step 2: Form a Square Operator**

Convert to a stable matrix using:

$$\mathbf{A} = \mathbb{E}_{B,T}[\mathbf{A}_{\text{bar}}]^\top \mathbb{E}_{B,T}[\mathbf{A}_{\text{bar}}]$$

Evaluation Metrics

Similarity Measures

Geometric

Frobenius Distance

$$\|A - H\|_F$$

Difference in operator magnitude

Cosine Similarity

$$\frac{\langle A, H \rangle}{\|A\|_F \|H\|_F}$$

Directional alignment in operator space

Spectral

Projection onto HiPPO Space

$$\|\text{Proj}_{\mathcal{H}}(A)\|_F$$

Component explainable by HiPPO operators

Residual Error

$$\|A - \text{Proj}_{\mathcal{H}}(A)\|_F$$

Non-HiPPO learned dynamics

Metrics evaluated across 3 datasets: DNA Sequences, Selective Copying, and Induction Heads

Key Findings

Mamba Aligns with LagT Dynamics

- **Frobenius Distance:**
LagT ~ 11.9 vs LegS/T ~ 178
- **Cosine Similarity:**
LagT ~ 0.71 (Highest Alignment shown by it)
- **Projection:**
Weak across all bases ($\sim 10^{-3}$)
- **Stability:**
Consistent across layers and all 3 datasets:
DNA Sequences, Selective Copying, Induction Heads

Core Insight

Mamba behaves more like a data-adapted Laguerre (LagT) system rather than LegS, even when the matrix is initialized using LegS operators.

From Hypothesis to Experiment

Guiding Mamba Towards LagT

- **Experimental Direction:**

To introduce an additional regularization term in the loss to encourage alignment between learned Mamba matrices and LagT structure.

- **Key Idea:**

Instead of learning dynamics entirely from scratch, softly bias the model toward a known optimal memory system.

- **Expected Outcome:**

- Reduced number of epochs to reach comparable performance

HiPPO LagT Regularization Setup

Injecting Structural Priors

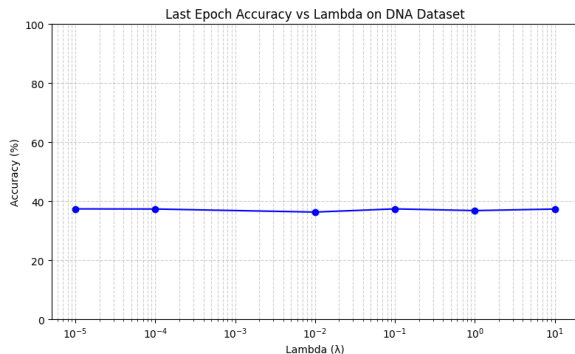
Modified Objective

$$\mathcal{L} = \mathcal{L}_{CE} + \lambda \mathcal{L}_{HiPPO}$$

- **Implementation Detail:** Inject LagT diagonals into the loss term to guide matrix evolution.
- **Training Protocol:** Evaluate across varying penalty weights (λ).

Regularization Results (DNA Dataset)

Effect of LagT Regularization



Dataset: DNA Genome
Epochs: 20

Accuracy vs λ

λ	Accuracy (%)
10^{-5}	37.40
10^{-4}	37.36
10^{-2}	36.32
10^{-1}	37.40
10^0	36.83
10^1	37.34

Results of Regularization

Does Structure Help?

Empirical Observations:

- No significant improvement in validation accuracy across datasets.
- Convergence speed remained largely unchanged.
- Higher regularization did not consistently stabilize or enhance learning.

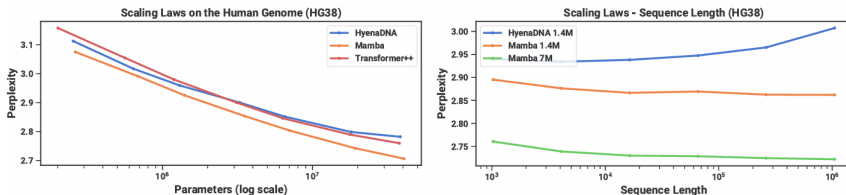
Key Insight

Mamba inherently learns efficient structured dynamics; explicit LagT regularization offers minimal additional benefit.

Problem 3: Mamba vs Convolutions

Initial Verification of Mamba on Constrained Resources

The graph given in the paper:

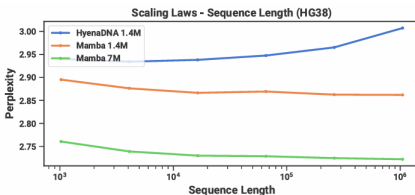
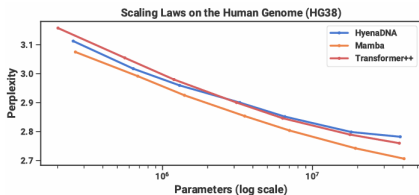


We set the sequence length to $2^{10} = 1024$ similar to the paper and trained the following models:

Micro-Scale Architecture	Number of Parameters	Validation PP (seq_len = 1024)
Mamba ($d_{model} = 64$)	33,285	2.8645
Mamba ($d_{model} = 128$)	117,765	2.7971

Initial Verification of Mamba

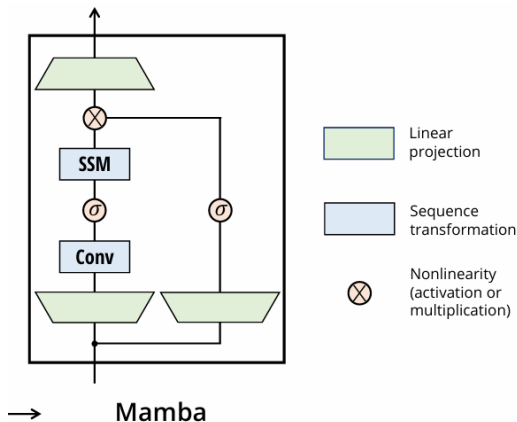
The graph given in the paper:



Sequence Length ($d_{model} = 128$)	Validation PP	Observation
256	2.8370	Baseline
1024	2.7904	Decreases
2048	2.8861	Increases drastically
4096	2.8157	Decreases again

Architecture: Pure Mamba (State Space Model)

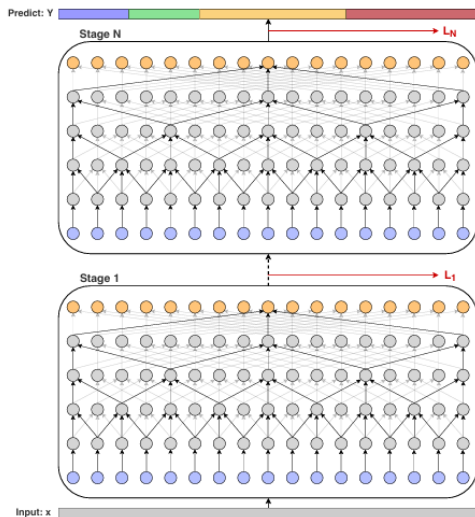
Mamba transitions away from attention mechanisms by using an LSSL along with a hardware aware Selective Scan Algorithm, allowing for linear time complexity $O(N)$.



Mamba Block Structure

Architecture: Pure TCN (Temporal Convolution)

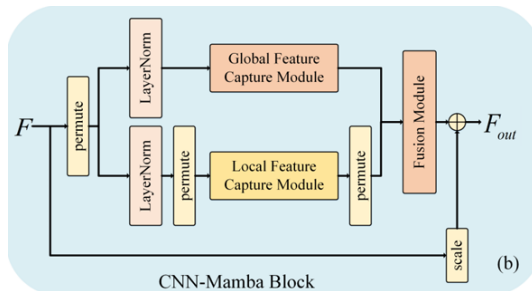
Temporal Convolutional Networks (TCN) are a special type of CNNs that use dilations and stacked layers to collect the information distributed along a sequence.



TCN Blocks

Architecture: Parallel Hybrid (Mamba + TCN)

Combine the global, long-range context of State Space Models with the local, high-frequency feature extraction of CNNs.



Experiment 1: sCIFAR & Parameter Constraints

The sCIFAR Dataset: Sequential CIFAR-10 flattens 32×32 images of the original CIFAR-10 dataset.

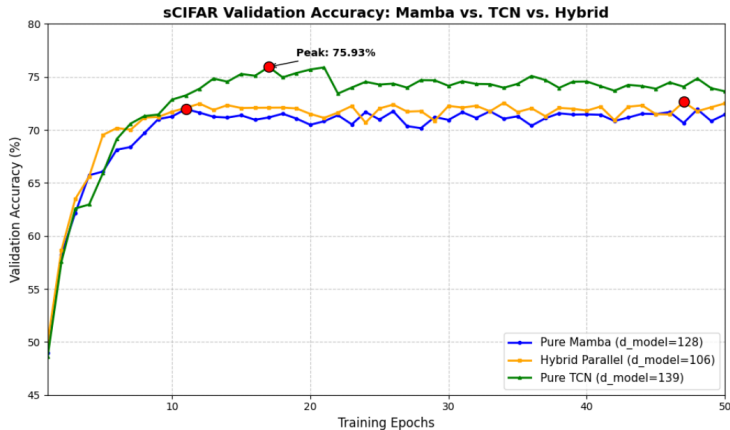
Methodology: Strict Parameter Budgeting

Architecture	TCN layers/ Mamba Blocks	<i>d_model</i>	Total Parameters
Pure Mamba	4	128	468,746
Pure TCN	8	139	468,996
Hybrid Parallel	4	106	465,774

Experiment 1: Results & Analysis

Peak Validation Accuracy:

- **Pure TCN:** 75.93%
- **Hybrid:** 72.63%
- **Pure Mamba:** 71.96%



Experiment 2: SpeechCommands & Temporal Capacity

The SpeechCommands Dataset: We truncated the audio waveform to a 4000 sequence long sequence to classify (35 classes).

Methodology: Parameter Budget & Width Restriction

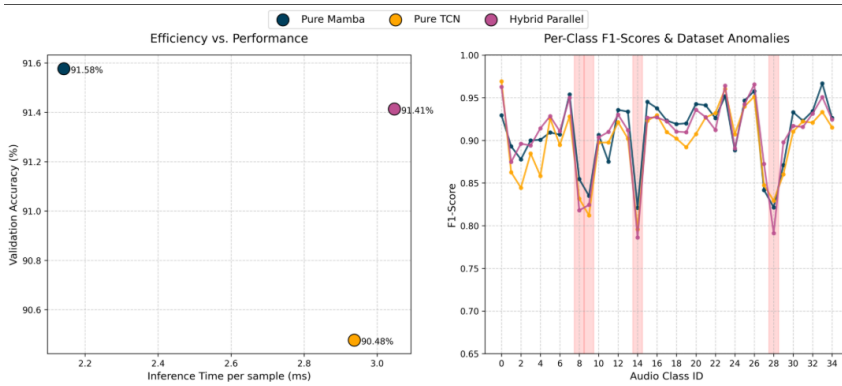
To maintain a strict parameter budget for fair comparison similar to experiment 1, the parameters were kept equal for all 3 models. This was again done through the manipulation of the hidden dimension.

Architecture	Layers / Blocks	d_{model}	Total Parameters
Pure Mamba	8	64	264,547
Pure TCN	11	89	267,658
Hybrid Parallel	8	53	258,940

Experiment 2: Results & Analysis

Peak Accuracy & Speed:

- **Pure Mamba:** 91.58% (2.05 ms/sample)
- **Hybrid:** 91.41% (2.97 ms/sample)
- **Pure TCN:** 90.48% (2.92 ms/sample)



U-Net Architecture for Image Defocus Deblurring

We took inspiration from the architecture proposed in the Dual-Branch CNN-Mamba Method for Image Defocus Deblurring.

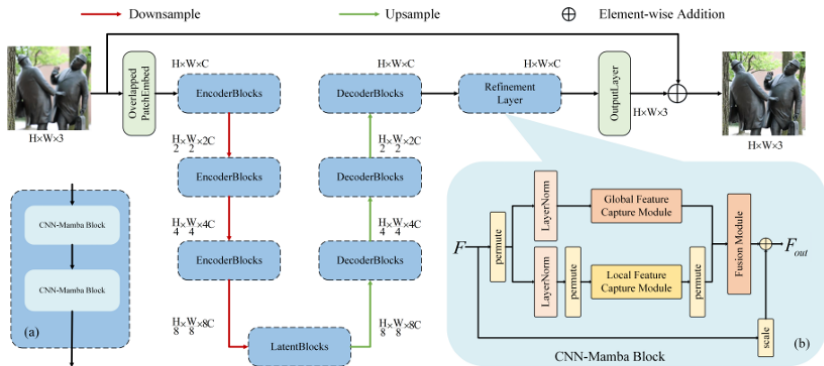


Figure 3. Architecture of CMDDNet for image defocus deblurring. CMDDNet adopts the classic U-Net framework. The encoder, latent layer, and decoder are constructed by stacking two CNN-Mamba blocks as illustrated in (a). The structure of each CNN-Mamba block is depicted in (b).

Modified Architectural Design & Ablation

Modular architecture

To test the performance and necessity of the dual-branch approach, we implemented a custom U-Net capable of dynamically swapping its core computational blocks:

- **Pure CNN Block:** Standard bidirectional convolutions.
- **Pure Mamba Block:** Flattened 1D sequence processing.
- **CNN Mamba (Hybrid) Block:** The dual pathway fusion.

Note that we use just a single block per `EncoderBlocks` and `DecoderBlocks`, as opposed to 2 blocks in serial as shown in the paper.

Experiment 3: The DPDD dataset, validation metrics and parameter budgeting

The DPDD Dataset: The Dual-Pixel Defocus Deblurring dataset provides heavily blurred source images mapped to perfectly sharp targets.

Validation Metrics:

- **PSNR:** Peak Signal to Noise Ratio.
- **LPIPS:** Learned Perceptual Image Patch Similarity.

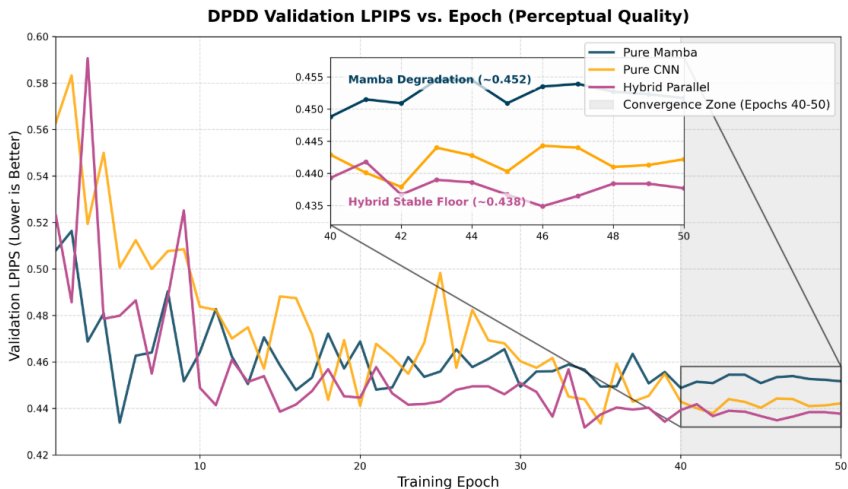
Methodology: Strict Parameter Budgeting

Architecture	<i>base_c</i>	Parameters
Pure CNN	88	4,314,467
Pure Mamba	64	4,286,659
Hybrid Parallel	56	4,325,723

Experiment 3: Results & Analysis

Metrics and Comparison:

- Mamba wins on peak PSNR (22.29). Hybrid (22.21) and CNN (21.86).
- Hybrid wins stable LPIPS floor (~ 0.438) and peak LPIPS.



Problem 4: Edge CNN-Mamba Deployment

CNN-Mamba Model

Experimenting with CNN-Mamba Architectures at Variable Depths for Edge Deployment

- **The Objective:** To prove that a hybrid CNN-Mamba architecture can achieve real-time, highly efficient human motion tracking on power-constrained edge robotics.

Dataset

Dataset Selection: The UCF101 Benchmark

- **The Scope:** 13,320 video clips categorized into 101 distinct human action classes.
- **The Split:** Standard Train/Test distribution ($\sim 9,500$ training videos / $\sim 3,800$ validation videos).
- **Why UCF101?:**
 - **Accelerated Training Cycles:** The smaller total video volume drastically reduced training times.



The Proposed CNN-Mamba Architecture

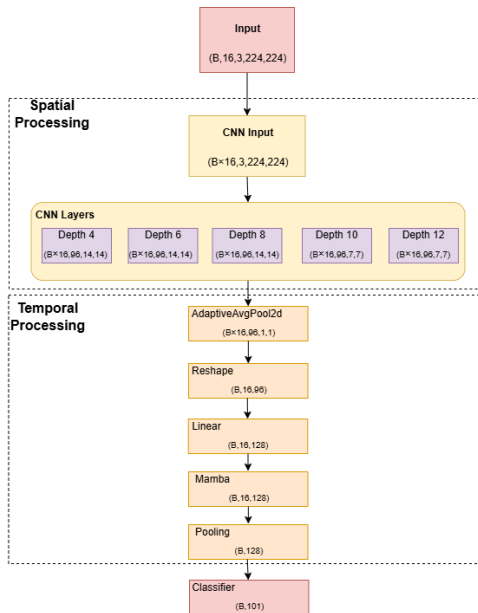


Figure: Overview of the Spatial (CNN) to Temporal (Mamba) Handoff

Experimental Setup & Methodology

The Edge Physical Test

- **Target:** NVIDIA Jetson Orin Nano
- **Power States:** **15W** (Performance) vs. **7W** (Low-Battery/Power-Saving)

The Test Variables: 5 discrete CNN-Mamba Architectures

- Varied by Handoff Depths: {4, 6, 8, 10, 12}

The Environment: Continuous end-to-end inference using a 720p webcam for 30 minutes.

The Metrics Recorded:

- **Speed:** Raw Throughput (FPS) and End-to-End Latency (ms)
- **Performance:** Predictive Accuracy (Top-1, Top-5, Top 10 classes, Top 50 classes)

Results: Accuracy & Subset Performance

Handoff Index	Top-1 (%)	Top-5 (%)	Top-10 Avg (%)	Top-50 Avg (%)
Depth 4	31.69	56.23	79.1	51.5
Depth 6	36.43	61.01	79.6	56.9
Depth 8	38.86	65.93	82.6	60.3
Depth 10	50.96	78.69	90.0	72.7
Depth 12	56.46	79.36	91.6	78.0

Table: Comparison of global accuracy vs. top-performing class subsets.

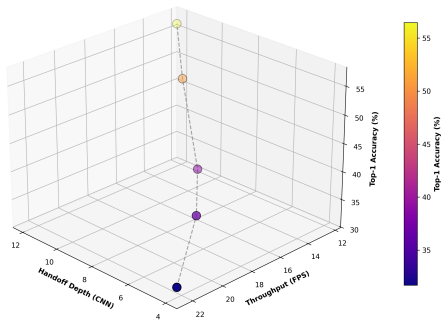
Overall Model Limitations & Trade-offs

- **The Primary Limitation: Small objects and movements**
 - Writing on Board, Playing Flute, Brushing Teeth.
- **The Performance Anchor: High movement activities**
 - Biking, Hula Hoop, Body-Weight Squats.

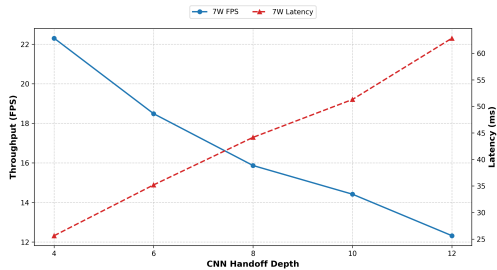
Key Insight: *The model trades fine-grained object detection for high-speed, hardware-efficient motion tracking.*

Hardware Evaluation: 7W Power-Saving Mode

7W Power Mode: Depth vs FPS vs Accuracy

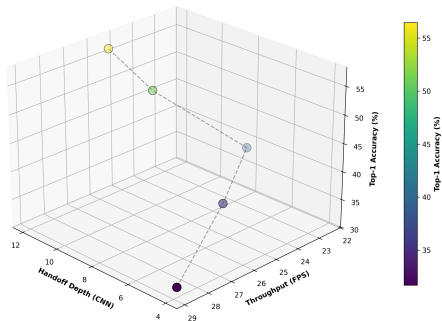


7W Power Mode: Handoff vs FPS & Latency

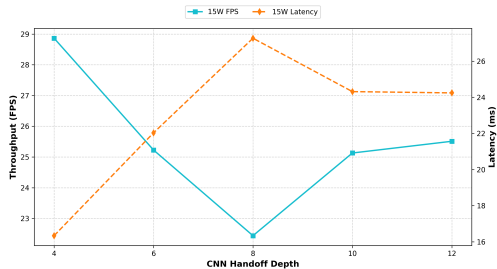


Hardware Evaluation: 15W Performance Mode

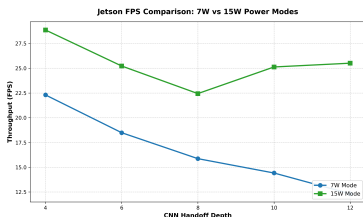
15W Power Mode: Depth vs FPS vs Accuracy



15W Power Mode: Handoff vs FPS & Latency



Conclusion: Adaptive Edge Deployment Strategy



- **High-Power State (15W):**

- **Behavior:** Prefer **Depth 12** when battery is fully charged or plugged in.
- **Result:** Achieves **91.6% accuracy** on top 10 core dynamic actions at real-time speeds.

- **Low-Power State (7W):**

- **Behavior:** Prefer **Depth 8** when operating under strict battery limits.
- **Result:** Extends operational battery life while maintaining a robust **82.6% accuracy** on core actions.

Future Scope

- **Optimization:** Diagnosing the hardware-level **FPS drop anomaly** at Depth 8 to ensure linear performance scaling.
- **Pre-training:** Implementing a **Kinetics-400** pre-training pipeline prior to UCF101 fine-tuning.
 - **Hypothesis:** Broadening the initial feature set will force the network to retain critical micro-motions, boosting global accuracy.

Thank You



References

References I



Hochreiter, S., & Schmidhuber, J. (1997).

Long short-term memory.

Neural Computation, 9(8), 1735–1780.



Cho, K., et al. (2014).

Learning phrase representations using RNN encoder-decoder for statistical machine translation.

arXiv preprint arXiv:1406.1078.



Bai, S., Kolter, J. Z., & Koltun, V. (2018).

An empirical evaluation of generic convolutional and recurrent networks for sequence modeling.

arXiv preprint arXiv:1803.01271.



Vaswani, A., et al. (2017/2023).

Attention is all you need.

arXiv preprint arXiv:1706.03762.



Gu, A., Dao, T., Ermon, S., Rudra, A., & Ré, C. (2020).

HiPPO: Recurrent memory with optimal polynomial projections.

arXiv preprint arXiv:2008.07669.



Gu, A., et al. (2021).

Combining recurrent, convolutional, and continuous-time models with linear state-space layers.

arXiv preprint arXiv:2110.13985.



Gu, A., Goel, K., & Ré, C. (2022).

Efficiently modeling long sequences with structured state spaces (S4).

arXiv preprint arXiv:2111.00396.

References II



Smith, J. T. H., Warrington, A., & Linderman, S. W. (2023).
Simplified state space layers for sequence modeling (S5).
arXiv preprint arXiv:2208.04933.



Gu, A., & Dao, T. (2024).
Mamba: Linear-time sequence modeling with selective state spaces (S6).
arXiv preprint arXiv:2312.00752.



Wang, B., et al. (2025).
Dual-Mamba: A hybrid CNN-Mamba architecture for tumor segmentation from 3D medical images.
47th Annual International Conference of the IEEE EMBC, 1–7.



He, H., et al. (2024).
MobileMamba: Lightweight multi-receptive visual Mamba network.
arXiv preprint arXiv:2411.15941.



Luo, F., et al. (2025).
ActivityMamba: A CNN-Mamba hybrid neural network for efficient human activity recognition.
IEEE Transactions on Mobile Computing, 24(8), 6797–6811.



Vakilipourtakalou, P., & Mou, L. (2020).
How chaotic are recurrent neural networks?
arXiv preprint arXiv:2004.13838.



Chattopadhyay, A., Hassanzadeh, P., & Subramanian, D. (2020).
Data-driven predictions of a multiscale Lorenz 96 chaotic system using machine-learning methods.
Nonlinear Processes in Geophysics, 27(3), 373–389.

References III



Wang, Q., et al. (2024).

Chaotic time series prediction based on physics-informed neural operator.

Chaos, Solitons & Fractals, 186, 115326.



Hart, J. D. (2024).

Attractor reconstruction with reservoir computers: The effect of the reservoir's conditional Lyapunov exponents.

Chaos: An Interdisciplinary Journal of Nonlinear Science, 34(4), 043123.



Zhao, W., Wu, C., Lu, J., & Li, R. (2025).

Dual-branch CNN-Mamba method for image defocus deblurring.

Applied Sciences, 15(6), 3173.



Abu Farha, Y., & Gall, J. (2019).

MS-TCN: Multi-stage temporal convolutional network for action segmentation.

Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 12027–12035.

Appendix: Core Mathematical Formulations

Continuous-Time State-Space Representation

The Continuous-Time Formulation

The Linear State-Space Layer (LSSL) maps a 1-dimensional input function $u(t)$ to an output $y(t)$ through an implicit hidden state $x(t)$. It simulates a linear continuous-time state-space representation:

$$\dot{x}(t) = Ax(t) + Bu(t) \tag{1}$$

$$y(t) = Cx(t) + Du(t) \tag{2}$$

- $A \in \mathbb{R}^{N \times N}$ controls the continuous evolution of the system's memory.
- $B \in \mathbb{R}^{N \times 1}$, $C \in \mathbb{R}^{1 \times N}$, and $D \in \mathbb{R}^{1 \times 1}$ are projection parameters.

Dimensionality Note: For an input sequence with H features, the LSSL independently broadcasts H copies of this 1-D to 1-D mapping, creating an H -dimensional sequence model[cite: 5317].

Discretization via Generalized Bilinear Transform

Approximating the Integral Equation

To evaluate the system on digital hardware, the differential equation must be integrated over a discrete time step Δt_k .

$$x(t_k) - x(t_{k-1}) = \int_{t_{k-1}}^{t_k} Ax(s)ds + \int_{t_{k-1}}^{t_k} Bu(s)ds$$

The Generalized Bilinear Transform (GBT) approximates this integral using a convex combination of the left and right endpoints, weighted by a parameter α :

$$x_k - x_{k-1} \approx \Delta t_k [(1 - \alpha)Ax_{k-1} + \alpha Ax_k] + \Delta t_k Bu_k$$

Rearranging for x_k isolates the exact discrete-time update rule:

$$x_k = (I - \alpha \Delta t_k A)^{-1} (I + (1 - \alpha) \Delta t_k A) x_{k-1} + (I - \alpha \Delta t_k A)^{-1} \Delta t_k Bu_k$$

Specific Discretization Schemes (α values)

Depending on the choice of α , the GBT recovers classical numerical integration methods:

1. Forward Euler ($\alpha = 0$)

- Holds the left endpoint constant throughout the integral.
- Equation: $x_k = (I + \Delta t A)x_{k-1} + \Delta t Bu_k$

2. Backward Euler ($\alpha = 1$)

- Holds the right endpoint constant. This maps perfectly to the ubiquitous gating mechanisms in modern RNNs.
- Equation: $x_k = (I - \Delta t A)^{-1}x_{k-1} + (I - \Delta t A)^{-1}\Delta t Bu_k$

3. Bilinear Method / Trapezoid Rule ($\alpha = 1/2$)

- Takes the exact average of the endpoints. This is the primary method used by LSSLs because it preserves the mathematical stability of the continuous system.
- Equation: $x_k = (I - \frac{\Delta t}{2}A)^{-1}(I + \frac{\Delta t}{2}A)x_{k-1} + (I - \frac{\Delta t}{2}A)^{-1}\Delta t Bu_k$

The Convolutional View (Krylov Function)

Unrolling the Recurrence

Let $\bar{A}$ and $\bar{B}$ be the discretized matrices using the bilinear method ($\alpha = 1/2$)[cite: 5251, 5252]. The discrete-time state-space model is:

$$x_t = \bar{A}x_{t-1} + \bar{B}u_t \quad \text{and} \quad y_t = Cx_t + Du_t$$

Assuming an initial state of $x_{-1} = 0$, unrolling the recurrence for an arbitrary step k yields[cite: 5279, 5280]:

$$y_k = C(\bar{A})^k \bar{B}u_0 + C(\bar{A})^{k-1} \bar{B}u_1 + \dots + C\bar{A}\bar{B}u_{k-1} + C\bar{B}u_k + Du_k$$

The Krylov Convolutional Kernel

This unrolled sequence is mathematically equivalent to a non-circular continuous convolution[cite: 5281]:

$$y = \mathcal{K}_L(\bar{A}, \bar{B}, C) * u + Du$$

Where the filter $\mathcal{K}_L$ is the Krylov function of length L [cite: 5281]:

$$\mathcal{K}_L(A, B, C) = \left(C\bar{B}, C\bar{A}\bar{B}, \dots, C\bar{A}^{L-1}\bar{B} \right)$$

This demonstrates how an LSSL can process an entire sequence of length L in parallel during training via Fast Fourier Transforms (FFTs)[cite: 5284].

Online Function Approximation

The Optimization Objective

At every time step t , HiPPO seeks a polynomial $g^{(t)}$ of degree $< N$ that minimizes the approximation error of the history $f_{\leq t}$ with respect to a time-varying measure $\mu^{(t)}$:

$$\|f_{\leq t} - g^{(t)}\|_{L_2(\mu^{(t)})}$$

Continuous-Time Dynamics

The coefficients $c(t) \in \mathbb{R}^N$ of this optimal projection evolve over time as a linear differential equation:

$$\frac{d}{dt}c(t) = A(t)c(t) + B(t)f(t)$$

Where $A(t) \in \mathbb{R}^{N \times N}$ and $B(t) \in \mathbb{R}^{N \times 1}$ are determined entirely by the choice of the measure $\mu^{(t)}$.

Sliding Window Memory

- **Measure:** Translates a uniform window of fixed length θ across time:

$$\mu^{(t)}(x) = \frac{1}{\theta} \mathbb{I}_{[t-\theta, t]}(x)$$

- **LTI Dynamics:** The system simplifies to a Linear Time-Invariant (LTI) ODE:

$$\frac{d}{dt}c(t) = -\frac{1}{\theta}Ac(t) + \frac{1}{\theta}Bf(t)$$

Matrix Formulation (LMU Exact Recovery)

Using the specific orthonormal scaling $\lambda_n = (2n + 1)^{1/2}(-1)^n$, we recover the exact Legendre Memory Unit (LMU) updates:

$$A_{nk} = (2n + 1) \begin{cases} (-1)^{n-k} & \text{if } k \leq n \\ 1 & \text{if } k \geq n \end{cases}$$

$$B_n = (2n + 1)(-1)^n$$

HiPPO-LagT (Translated Laguerre)

Exponentially Decaying History

- **Measure:** Utilizes an exponentially decaying measure, prioritizing recent history without a hard cutoff:

$$\mu^{(t)}(x) = e^{-(t-x)} \mathbb{I}_{(-\infty, t]}(x)$$

- **LTI Dynamics:** The system simplifies to a Linear Time-Invariant (LTI) ODE:

$$\frac{d}{dt}c(t) = -Ac(t) + Bf(t)$$

Matrix Formulation

The matrices for this parameter-free structure evaluate to:

$$A_{nk} = \begin{cases} 1 & \text{if } n \geq k \\ 0 & \text{if } n < k \end{cases}$$

$$B_n = 1$$

Note: This corresponds to the standard Laguerre basis where $\alpha = 0$ without additional tilting.

HiPPO-LegS (Scaled Legendre)

Timescale-Invariant Memory

- **Measure:** Scales the window over time to uniformly weight all history from $t = 0$ to current time t :

$$\mu^{(t)}(x) = \frac{1}{t} \mathbb{I}_{[0,t]}$$

- **Dynamics:** Because the measure scales with t , the resulting ODE features a time-varying $1/t$ dilation factor:

$$\frac{d}{dt}c(t) = -\frac{1}{t}Ac(t) + \frac{1}{t}Bf(t)$$

Matrix Formulation

The optimal coefficients for the scaled measure are derived as:

$$A_{nk} = \begin{cases} (2n+1)^{1/2}(2k+1)^{1/2} & \text{if } n > k \\ n+1 & \text{if } n = k \\ 0 & \text{if } n < k \end{cases}$$

$$B_n = (2n+1)^{1/2}$$

ODE Discretization (1/2): Euler Methods

From Continuous to Discrete Time

To process sequence data, the continuous LTI ODE $\frac{d}{dt}c(t) = Ac(t) + Bf(t)$ must be discretized with a step size Δt .

Forward Euler

Approximates the integral by evaluating the derivative at the left endpoint:

$$c(t + \Delta t) = (I + \Delta t A)c(t) + \Delta t B f(t)$$

Backward Euler

Approximates the integral by evaluating the derivative at the right endpoint:

$$c(t + \Delta t) = (I - \Delta t A)^{-1}c(t) + \Delta t (I - \Delta t A)^{-1}B f(t)$$

ODE Discretization (2/2): Advanced Methods

Bilinear Transform & Zero-Order Hold

Bilinear (Trapezoid Rule)

Averages the endpoints for better numerical stability:

$$c(t + \Delta t) = \left(I - \frac{\Delta t}{2} A \right)^{-1} \left(I + \frac{\Delta t}{2} A \right) c(t) + \Delta t \left(I - \frac{\Delta t}{2} A \right)^{-1} B f(t)$$

Zero-Order Hold (ZOH)

Calculates the exact solution assuming the input $f(t)$ remains constant between t and $t + \Delta t$:

$$c(t + \Delta t) = e^{\Delta t A} c(t) + A^{-1} (e^{\Delta t A} - I) B f(t)$$

HiPPO-LegS: Generalized Discretization

Timescale-Invariant Recurrences

For HiPPO-LegS, the linear ODE includes a dynamic $1/t$ dilation factor: $\frac{d}{dt}c(t) = \frac{1}{t}Ac(t) + \frac{1}{t}Bf(t)$.

Generalized Bilinear Transformation (GBT) for LegS

Applying GBT (where $\alpha \in [0, 1]$) yields a recurrence that is completely independent of the step size Δt :

$$c_{k+1} = \left(I - \frac{\alpha}{k+1}A\right)^{-1} \left(I + \frac{1-\alpha}{k}A\right) c_k + \frac{1}{k} \left(I - \frac{\alpha}{k+1}A\right)^{-1} Bf_k$$

- Setting $\alpha = 0$ recovers the simple Forward Euler recurrence: $c_{k+1} = (I - \frac{1}{k}A)c_k + \frac{1}{k}Bf_k$.
- Setting $\alpha = 0.5$ recovers the highly stable Bilinear update.

HiPPO-LegT: Generalized Discretization

Sliding Window Recurrence

Unlike LegS, the translated Legendre (LegT) ODE is $\frac{d}{dt}c(t) = -\frac{1}{\theta}Ac(t) + \frac{1}{\theta}Bf(t)$. The discretization relies heavily on the ratio of the step size Δt to the window length θ .

Discrete Update Rule (via GBT)

Applying the Generalized Bilinear Transform (with $\alpha \in [0, 1]$) yields:

$$c_{k+1} = \left(I + \alpha \frac{\Delta t}{\theta} A\right)^{-1} \left(I - (1 - \alpha) \frac{\Delta t}{\theta} A\right) c_k + \frac{\Delta t}{\theta} \left(I + \alpha \frac{\Delta t}{\theta} A\right)^{-1} B f_k$$

- **Timescale Sensitivity:** The accuracy of this recurrence degrades significantly if θ does not appropriately match the sequence length or if Δt is mis-specified.
- **LMU Connection:** Setting $\alpha = 0$ (Forward Euler) provides the simplest sliding-window update for the Legendre Memory Unit.

HiPPO-LagT: Generalized Discretization

Exponential Decay Recurrence

The translated Laguerre (LagT) ODE simplifies to $\frac{d}{dt}c(t) = -Ac(t) + Bf(t)$, making its discretization straightforward but dependent on Δt .

Discrete Update Rule (via GBT)

Applying the Generalized Bilinear Transform yields:

$$c_{k+1} = (I + \alpha \Delta t A)^{-1} (I - (1 - \alpha) \Delta t A) c_k + \Delta t (I + \alpha \Delta t A)^{-1} B f_k$$

- **Forward Euler ($\alpha = 0$):** Reduces to $c_{k+1} = (I - \Delta t A) c_k + \Delta t B f_k$.
- **RNN Gating Connection:** When constrained to $N = 1$, this equation precisely mirrors the mathematical foundation of standard gating mechanisms (like LSTMs and GRUs), where the state is updated via $(1 - \Delta t) c_k + \Delta t f_k$.

The S5 Layer Architecture

The S4 Paradigm: Independent SISO

- S4 uses a bank of H independent Single-Input, Single-Output (SISO) linear SSMs.
- Each SSM processes one dimension of the input sequence independently.
- **Bottleneck:** Requires an external position-wise linear mixing layer to combine features at the end.

The S5 Paradigm: Dense MIMO

- S5 collapses the bank into a single Multi-Input, Multi-Output (MIMO) SSM.
- Uses a dense input matrix $B \in \mathbb{C}^{P \times H}$ and output matrix $C \in \mathbb{C}^{H \times P}$.
- **Advantage:** Inherent feature mixing. The latent size P can be significantly smaller than the $H \times N$ effective size of S4, maintaining identical computational complexity.

S5's Parallel Scan Engine

The S4 Convolution Limitation

S4's offline speed relies on converting the discrete recurrence into a continuous convolution, calculating a Cauchy kernel, and applying FFTs. **Limitation:** This strictly requires regularly spaced observations and time-invariant systems.

The S5 Solution: Associative Parallel Scans

S5 remains entirely in the recurrent time-domain by utilizing a parallel associative scan over the elements $c_k = (\bar{A}, \bar{B}u_k)$ using the binary operator:

$$q_i \bullet q_j := (q_{j,a} \odot q_{i,a}, \quad q_{j,a} \otimes q_{i,b} + q_{j,b})$$

- **Speed:** Reduces sequential operations to an $\mathcal{O}(\log L)$ parallel time complexity.
- **Flexibility:** Because it remains recurrent, S5 natively handles variable observation intervals (irregular sampling) by injecting a unique Δ_t at every step.

S5 Diagonalized Parameterization

1. Continuous-Time Diagonalization

To enable the $\mathcal{O}(P \log L)$ parallel scan, S5 must diagonalize the state matrix $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$, where $\mathbf{\Lambda}$ contains the eigenvalues. The latent dynamics are rewritten as:

$$\frac{d(\mathbf{V}^{-1}\mathbf{x}(t))}{dt} = \mathbf{\Lambda}\mathbf{V}^{-1}\mathbf{x}(t) + \mathbf{V}^{-1}\mathbf{B}u(t)$$

2. The Reparameterized System

By defining the new state $\tilde{\mathbf{x}}(t) = \mathbf{V}^{-1}\mathbf{x}(t)$ and projecting the input/output matrices ($\tilde{\mathbf{B}} = \mathbf{V}^{-1}\mathbf{B}$, $\tilde{\mathbf{C}} = \mathbf{C}\mathbf{V}$), the continuous system becomes perfectly diagonalized:

$$\frac{d\tilde{\mathbf{x}}(t)}{dt} = \mathbf{\Lambda}\tilde{\mathbf{x}}(t) + \tilde{\mathbf{B}}u(t), \quad y(t) = \tilde{\mathbf{C}}\tilde{\mathbf{x}}(t) + \mathbf{D}u(t)$$

3. Zero-Order Hold (ZOH) Discretization

This diagonal continuous system is then discretized using a timescale parameter Δ to yield the final discrete recurrence parameters dropped into the parallel scan:

$$\bar{\mathbf{A}} = e^{\mathbf{\Lambda}\Delta}, \quad \bar{\mathbf{B}} = \mathbf{\Lambda}^{-1}(\bar{\mathbf{A}} - \mathbf{I})\tilde{\mathbf{B}}, \quad \bar{\mathbf{C}} = \tilde{\mathbf{C}}, \quad \bar{\mathbf{D}} = \mathbf{D}$$

The Initialization Trade-off (HiPPO-N)

The Diagonalization Barrier

To achieve $\mathcal{O}(P \log L)$ parallel scan complexity, the state matrix A must be diagonalized ($A = V\Lambda V^{-1}$).

The analytical HiPPO-LegS matrix is not stably diagonalizable.

The S5 Normal Approximation

- **The Fix:** S5 initializes its state matrix using HiPPO-N, the diagonalizable normal component of the LegS matrix.
- **The Proof:** As the state dimension $N \rightarrow \infty$, the ODE defined by HiPPO-N converges identically to the dynamics of the full HiPPO-LegS matrix.
- **Result:** S5 achieves comparable long-horizon stability (e.g., LRA Path-X 98.5% accuracy) while satisfying the strict diagonalization requirements of the parallel scan.

Continuous-Time State Space Model

System Equations

$$h'(t) = Ah(t) + Bx(t)$$

$$y(t) = Ch(t)$$

Variables

- $x(t)$: 1D input function
- $h(t)$: N -dimensional latent state
- $y(t)$: 1D output sequence
- A : Structured $N \times N$ matrix
- B, C : Projection parameters

Discretization & Recurrence

Zero-Order Hold (ZOH)

- $\bar{A} = \exp(\Delta A)$
- $\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B$

Discrete Recurrent Computation

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t$$

$$y_t = Ch_t$$

- Δ : Step size for transformation
- h_t : Hidden state at discrete timestep t

Mamba Selection Mechanism

To make the model "selective," parameters become functions of the input x_t :

$$B_t = \text{Linear}_N(x_t)$$

$$C_t = \text{Linear}_N(x_t)$$

$$\Delta_t = \text{softplus}(\text{Parameter} + \text{Linear}(x_t))$$

- **Context-Aware:** B_t , C_t , Δ_t vary across the sequence.
- **Softplus:** Ensures Δ_t remains positive.
- **Linear_N:** Projects input to dimension N .

Connection to Gating

For $N = 1, A = -1, B = 1$, selective recurrence simplifies to a Gated Linear Unit:

Selective Recurrence as Gating

$$g_t = \sigma(\text{Linear}(x_t))$$

$$h_t = (1 - g_t)h_{t-1} + g_tx_t$$

- g_t : Input-dependent gate (Sigmoid σ).
- **Mechanism**: Controls the trade-off between "remembering" h_{t-1} and "incorporating" new input x_t .